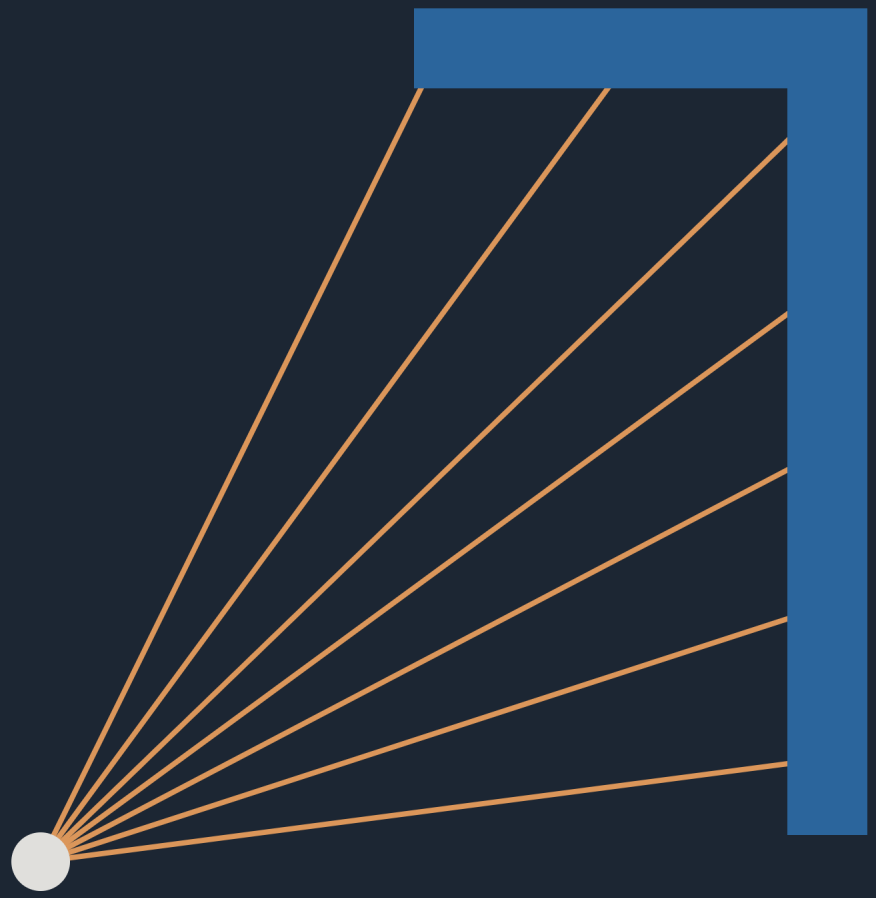
42 Tokyo ・ 評価用の説明資料

cub3D

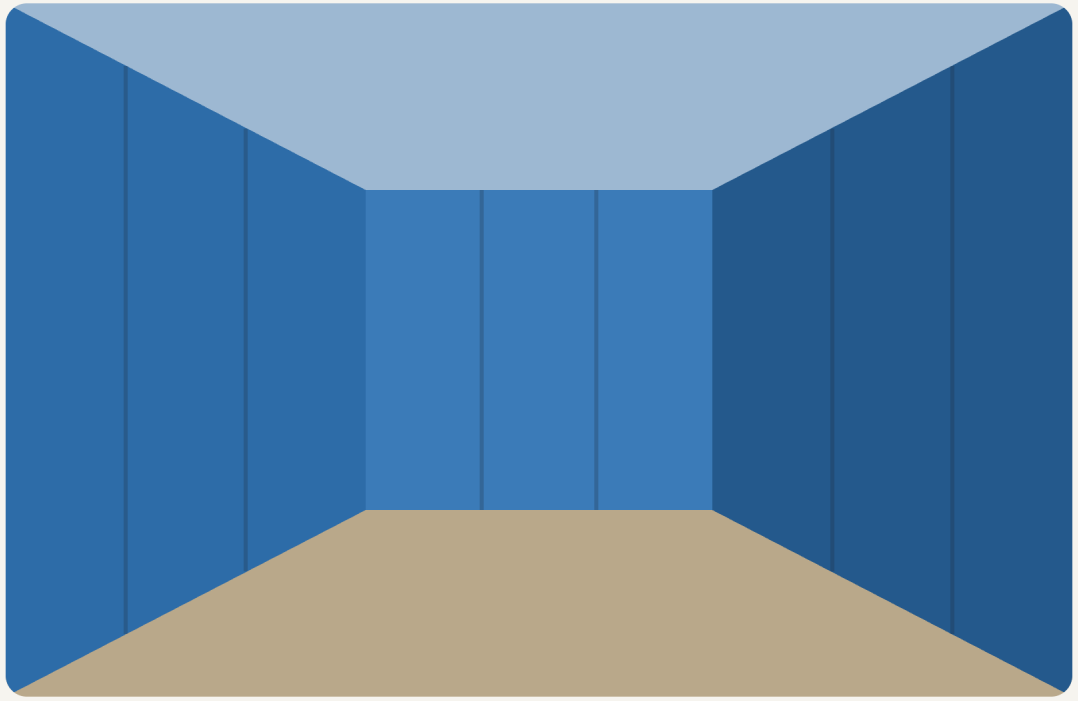
レイキャスティングで作る、3D の迷路

nkojima ・ tmase



cub3D とは

迷路の中を、一人称視点で歩くプログラム



- 「.cub」ファイルから迷路を読み込む
- 壁・床・天井を、奥行きのある 3D っぽい絵で描く
- 壁には東西南北で違う画像を貼る
- W A S D で移動、← → で向きを変える
- ESC か × ボタンで終了

cub3D とは

.cub ファイルに、迷路の設定を書く

```
NO ./textures/north.xpm
SO ./textures/south.xpm
WE ./textures/west.xpm
EA ./textures/east.xpm

F 220,100,0
C 225,30,0

111111
100101
101001
1100N1
111111
```

壁の画像　NO・SO・WE・EA

北・南・西・東の壁に貼る画像（xpm）のパス

床と天井の色　F・C

R,G,B を 0〜255 の数字3つで書く

マップ（いちばん最後）

1文字が1マス。1＝壁、0＝床、N・S・E・W＝プレイヤーの位置と向き。周りは壁で閉じている必要がある

maps/good/simple.cub

全体の流れ

読む → 計算する → 描く

.cub ファイル

迷路の設定

→

パーサー

設定を読んで、画像・色・マップを構造体に入れる

`parser/`

→

レイキャスト

画面の列ごとに視線（レイ）を飛ばして、壁までの距離を求める

`raycast/`

→

描画

天井・床・壁を1枚の画像に描いて、ウィンドウに貼る

`render/`

▷ **キーが押されるたびに**　位置と向きを更新 → レイキャスト → 描き直す

パーサー

.cub を読んで、3つのことをする

1

設定の行を読む

NO SO WE EA F C の6種類。順番は自由、空行も OK。重複・足りない・書き方がおかしい → Error

parse_config.c
parse_color.c

2

マップを配列にする

短い行の右をスペース（·）で埋めて、同じ長さにそろえる

```
1111111      1111111
10001    →   10001··
11111        11111··
```

parse_map.c
parse_map_array.c

3

壁で閉じているか調べる

閉じていないと、レイがマップの外に出てしまう → Error

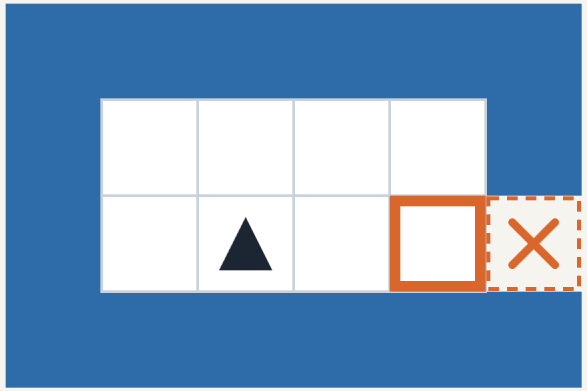
parse_map_wall.c

パーサー

壁で閉じているかは「上下左右」を見る

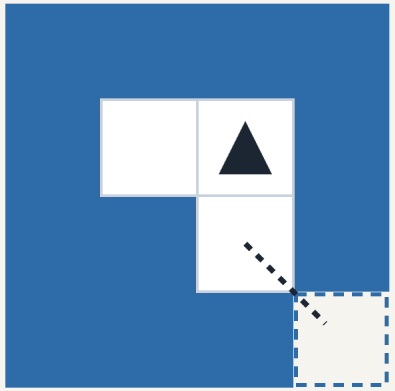
歩けるマス（0 と N・S・E・W）の上下左右のどれかが「マップの外」か「スペース」なら Error

NG：壁に穴がある



枠の床の右がマップの外 → Error

OK：角だけ欠けている



斜めは調べない。レイも移動も、縦か横に1マスずつしか進まないので、斜めの隙間からは外に出られない

parser/parse_map_wall.c — is_map_closed

パーサー

おかしい .cub は、Error を出して終わる

```
$ ./cub3D maps/bad/
  map_hole_top.cub
Error
Invalid map
```

1行目は必ず「Error」、2行目に理由

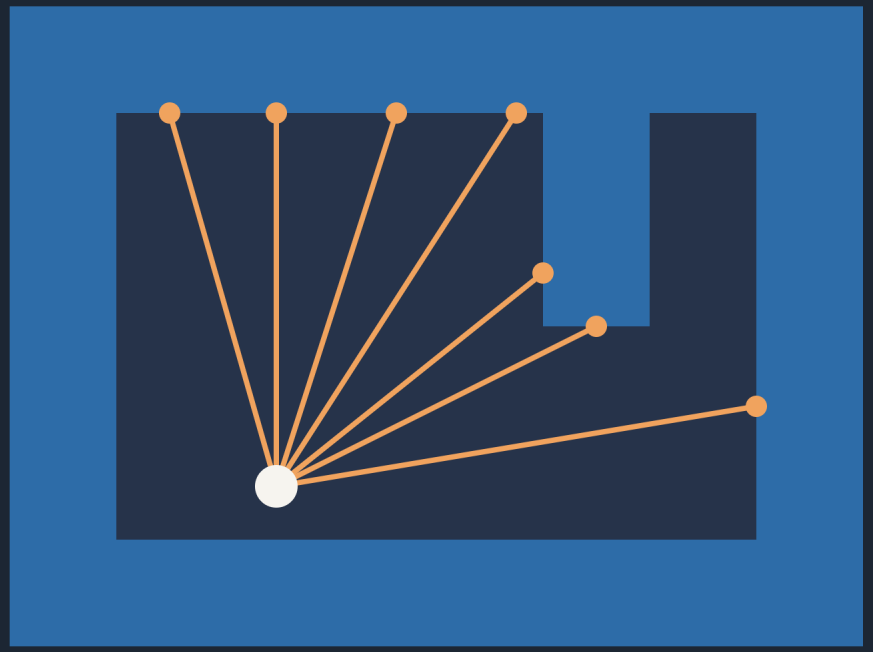
こういうとき	2行目のメッセージ
引数が1つじゃない、.cub じゃない	args error
ファイルが開けない	Failed to open file
設定の行がおかしい（足りない・書き方・色）	Invalid color code or texture path
マップがおかしい（文字・プレイヤー・壁）	Invalid map
画像が読めない（描画側でチェック）	failed to load texture: パス

Error で終わるときも、確保したメモリはすべて解放してから終了する

exit/cub_error.c — handle_error ・ parser/parse.c — parse

レイキャスティング

画面の横1列ごとに、レイを1本飛ばす



レイ＝プレイヤーの目から飛ばす、まっすぐな視線

画面の幅は 640 列 → レイも 640 本

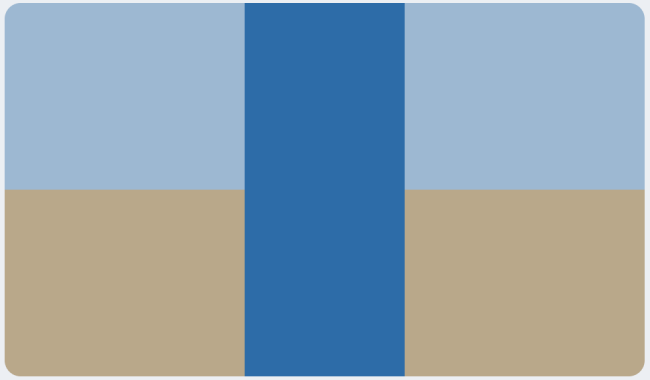
1本ずつ、壁に当たるまでの**距離**を測る

近い壁は高く、遠い壁は低く描く

レイキャスティング

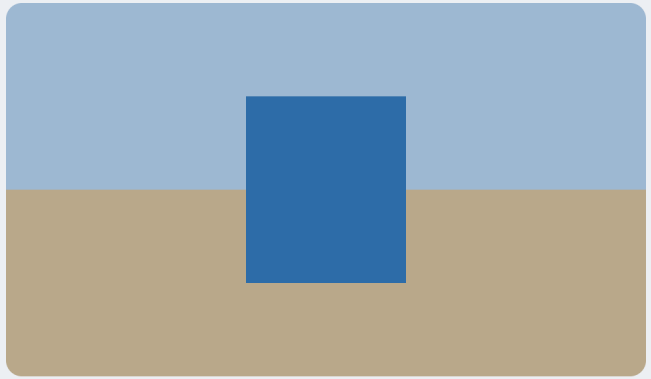
壁の高さは、距離で割るだけ

壁の高さ ＝ 画面の高さ ÷ 壁までの距離



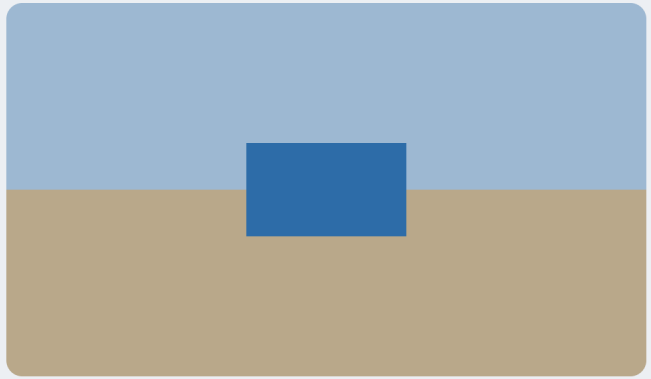
距離 1

画面の高さぴったり



距離 2

半分の高さ



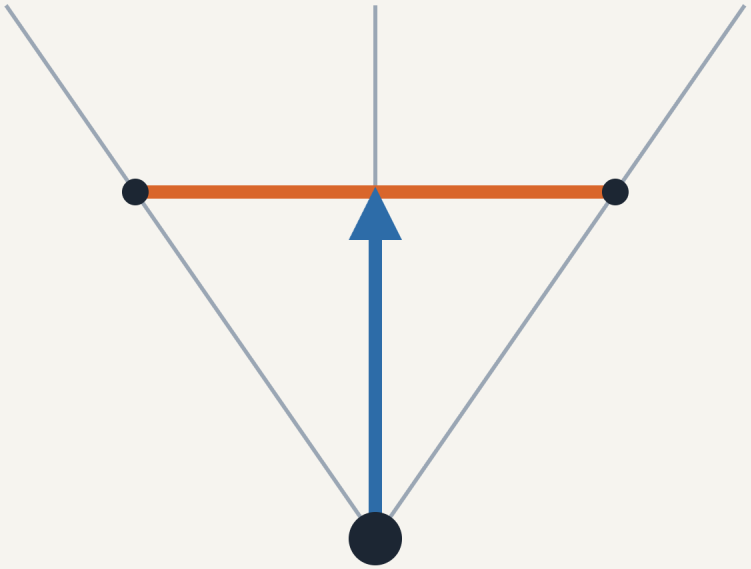
距離 4

4分の1の高さ

`render/render_wall.c — render_draw_column`

レイキャスティング

レイの方向は、プレイヤーの向きと視野で決まる



dir（青い矢印）：プレイヤーが向いている方向

plane（オレンジの線）：視野の左右の広がり。「カメラ平面」と呼ぶ

```
レイ = dir + plane × camera_x
```

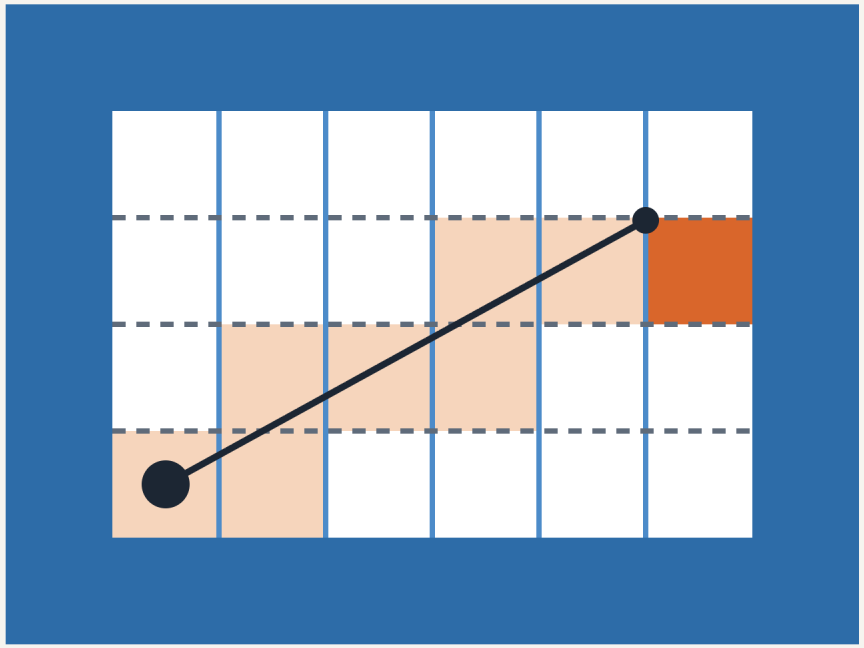
camera_x は画面の左端で −1、真ん中で 0、右端で +1

← → で回すときは、dir と plane を同じ角度だけ回す

raycast/raycast.c — raycast_init_ray · raycast/raycast_player.c — raycast_rotate

レイキャスティング

DDA：レイが通るマスを1つずつ調べる



レイは、マスとマスの**境目**を越えるたびに、隣のマスへ移る

次に来るのが**縦の境目（青い実線）**か**横の境目（灰色の点線）**か、近い方を越える

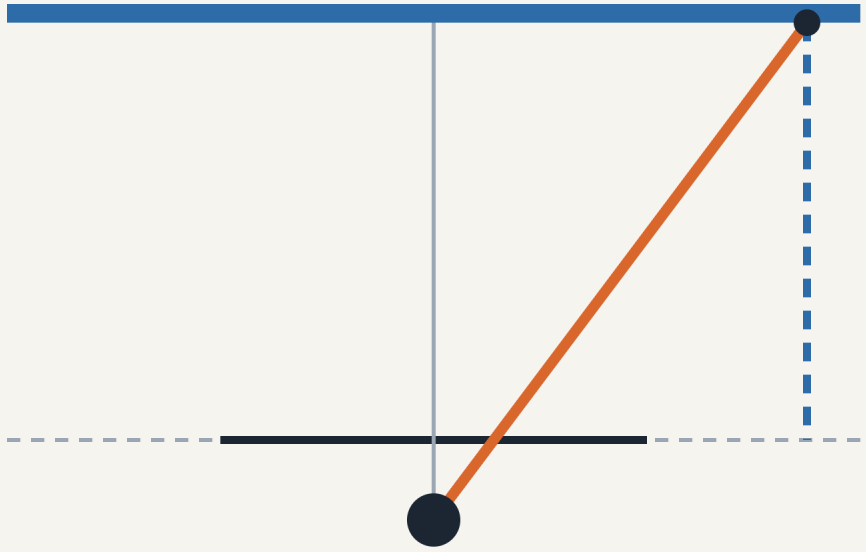
壁（1）のマスに入ったら止まる

最後に縦の境目を越えた → 東か西の面（side = 0）
最後に横の境目を越えた → 北か南の面（side = 1）

raycast/raycast.c — raycast_init_step ・ raycast_run_dda

レイキャスティング

壁の高さに使う距離は「垂直距離」



まっすぐの距離 ×

画面の端ほど長くなる → 平らな壁が丸く曲がって見える（魚眼）

垂直距離 ○

カメラ平面から壁までの距離 → 平らな壁は平らに見える

垂直距離は、DDA で壁を見つけたときに一緒に求まる

raycast/raycast.c — raycast_cast (perp_dist)

テクスチャ

どの画像を貼るか：越えた境目 × レイの向き

最後に越えた境目	レイが進む向き	貼る画像
横の境目（side 1）	北へ	NO
横の境目（side 1）	南へ	SO
縦の境目（side 0）	西へ	WE
縦の境目（side 0）	東へ	EA

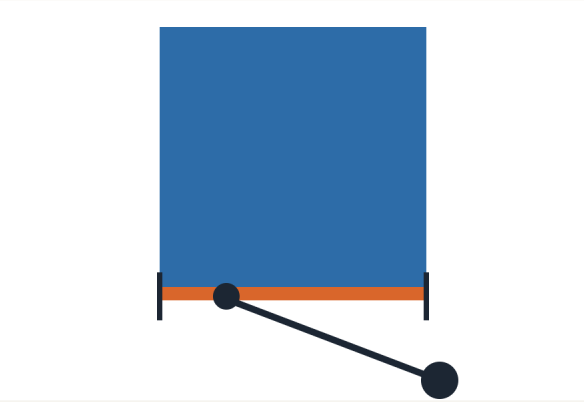


北に進むレイが当たる壁 ＝ 北を向いたときに見える壁 → **NO の画像**

`render/render_wall.c — render_pick_tex`

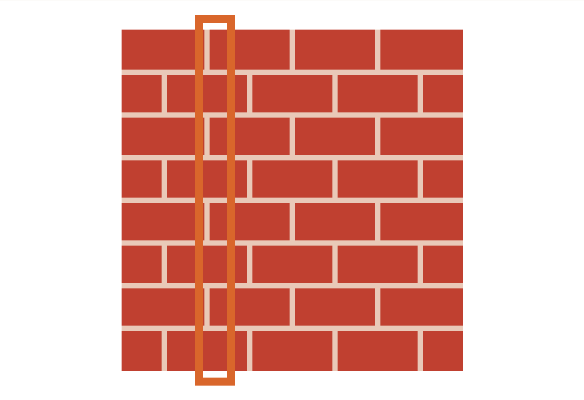
テクスチャ

当たった位置で、画像のどこを貼るかが決まる



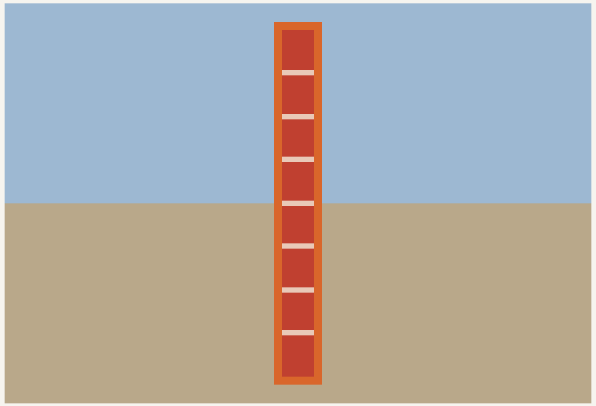
① 壁に当たった位置

壁の面の左端を 0、右端を 1 として、0.25 のところに当たった



② 画像の同じ位置の縦の帯

画像の左から 0.25 のところの、細い縦の帯を使う



③ 画面の1列に描く

その帯を、壁の高さまで縦に引き伸ばして描く

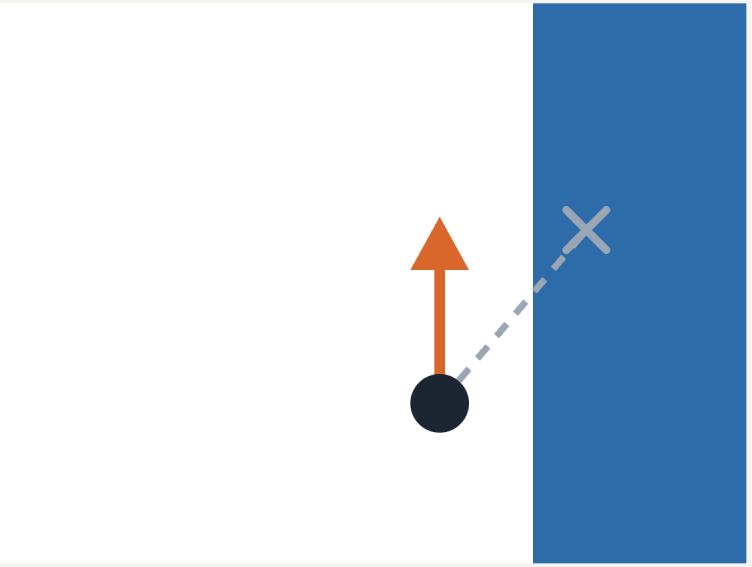
使う列 ＝ 当たった位置 × 画像の幅　（例：0.25 × 64 ＝ 16 列目）

南と西の壁だけは、そのままだと鏡みたいに左右が逆になるので、右から数え直す

`render/render_wall.c — render_tex_x ・ render_draw_column`

移動

x と y を別々に調べて、壁には入らない



W・S　前・後ろに進む

A・D　左・右に進む

←→　向きを回す

キーを押すたびに 0.1 マスずつ進む

進む先が壁なら止まる。**x と y を別々に**調べるので、斜めにぶつかっても**壁に沿って滑れる**

raycast/raycast_player.c — raycast_move